Purpose

VER-0 — verification/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/verification/VER-0_purpose.md`

---
library: verification
id: VER-0
schema_version: 1.0.0
status: draft

VER-0 — verification/ Purpose + Boundaries

Purpose
`verification/` defines Axion's **proof and completion system**:
- what "proof" is (types, required fields)
- how command runs are recorded (what was executed, outputs, status)
- how proofs are collected into a proof ledger
- what completion criteria mean (unit done + run done)

Verification is what makes gates like G7_VERIFICATION enforceable and audit-ready.

What it governs (in scope)
- Proof types registry (what proofs exist and how to interpret them)
- Proof ledger schema (the canonical record of proofs for a run)
- Command run tracking schema (what commands were executed, results, logs refs)
- Verification command policy hooks (what commands are allowed in which environments)
- Completion criteria (when a task is "done" and when a run is "done")

What it does NOT govern (out of scope)
- Gate DSL semantics → `gates/`
- Stage ordering and run manifest → `orchestration/`
- Policy meaning → `policy/`
- Adapter capability system plumbing → `system/`
- Ops standards (SLOs/alerts/etc.) → `ops/`

Consumers
- Gate evaluation stage (S8) for proof checks
- Orchestrator (pause/fail on missing proofs)
- Operator UI (proof viewer, command logs)

- Kit packager (include proof ledger in kit if required)

Determinism requirements
- Proof requirements are derived from pinned standards + plan acceptance evidence.
- Proof ledger is append-only for a run.
- Proof records reference artifacts/logs by path + optional hash.

Piece B — Boundary checklist

File: `Axion/libraries/verification/VER-0_boundary_checklist.md`

---
library: verification
id: VER-0a
schema_version: 1.0.0
status: draft

VER-0a — Boundary Checklist

Belongs in `verification/` if it answers:
- “What proofs are required and what do they look like?”
- “How do we record command execution and results?”
- “How do we decide when work/run is complete?”

Does NOT belong in `verification/` if it answers:
- “How do we evaluate gate predicates?” (gates)
- “What standards require proofs?” (standards)
- “How do we package the kit?” (kit)

Proof types registry

VER-1 — Proof types registry (what proofs exist)

Piece A — Proof type model (concept)

File: `Axion/libraries/verification/VER-1_proof_types_model.md`

```
---
library: verification
id: VER-1
schema_version: 1.0.0
status: draft
---

# VER-1 — Proof Types Model

## Purpose
Define the allowed proof types and their required fields so proof checking is deterministic.

## What a proof is
A proof is a structured record that a verification action occurred and produced evidence.

## Proof type examples
- command_run (a command executed, with logs/ref)
- test_suite (unit/integration/e2e results)
- lint_check (lint/format verification)
- build_artifact (successful build output)
- security_scan (SAST/DAST/dependency scan)
- manual_attestation (human approval for something)

## Required proof fields (minimum)
- proof_id
- proof_type
- status (pass/fail/warn)
- evidence_refs (paths to logs/artifacts)
- recorded_at
```

Piece B — Proof types registry schema (JSON Schema)

File: `Axion/libraries/verification/schemas/proof_types.v1.schema.json`

```
{
```

```
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "proof_types.v1",
  "type": "object",
  "required": ["registry_id", "schema_version", "types"],
  "properties": {
    "registry_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "types": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/proof_type" }
    }
  },

  "$defs": {
    "proof_type": {
      "type": "object",
      "required": ["proof_type", "required_fields", "description"],
      "properties": {
        "proof_type": {
          "type": "string",
          "enum": ["command_run", "test_suite", "lint_check", "build_artifact", "security_scan",
"manual_attestation"]
        },
        "description": { "type": "string", "minLength": 5 },
        "required_fields": { "type": "array", "items": { "type": "string" }, "minItems": 1 }
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Starter proof types registry

File: `Axion/libraries/verification/registries/proof_types.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "proof_types.v1",
```

```
  "types": [
    {
      "proof_type": "command_run",
      "description": "Evidence that a command was executed with recorded outputs/log refs.",
      "required_fields": ["command_run_id", "command", "exit_code", "logs_ref"]
    },
    {
      "proof_type": "test_suite",
      "description": "Evidence that a test suite ran and produced results.",
      "required_fields": ["suite", "passed", "failed", "results_ref"]
    },
    {
      "proof_type": "lint_check",
      "description": "Evidence that lint/format checks ran.",
      "required_fields": ["tool", "status", "logs_ref"]
    },
    {
      "proof_type": "build_artifact",
      "description": "Evidence that a build produced an artifact.",
      "required_fields": ["artifact_kind", "artifact_ref"]
    },
    {
      "proof_type": "security_scan",
      "description": "Evidence that a security scan ran (SAST/DAST/deps).",
      "required_fields": ["scan_type", "status", "report_ref"]
    },
    {
      "proof_type": "manual_attestation",
      "description": "Human attestation/approval for a requirement.",
      "required_fields": ["actor_id", "role", "statement", "expires_at"]
    }
  ]
}
```

Piece D — Determinism rules

File: `Axion/libraries/verification/VER-1_determinism_rules.md`

---
library: verification
id: VER-1a
schema_version: 1.0.0

status: draft

VER-1a — Determinism Rules (Proof Types)

- proof_type is a closed enum and must exist in the registry.
- required_fields are enforced by proof ledger validation.
- Registry is pinned per run (for reproducibility).

Piece E — Validation checklist

File: `Axion/libraries/verification/VER-1_validation_checklist.md`

---
library: verification
id: VER-1b
schema_version: 1.0.0
status: draft

VER-1b — Validation Checklist

- [ ] proof_types registry validates against schema
- [ ] proof_type values are unique and from the allowed enum set
- [ ] each proof_type has required_fields and description
- [ ] registry is pinned in the run manifest (or via pins)

Proof ledger schema

VER-1 — Proof types registry (what proofs exist)

Piece A — Proof type model (concept)

File: `Axion/libraries/verification/VER-1_proof_types_model.md`

```
---
library: verification
id: VER-1
schema_version: 1.0.0
status: draft
---

# VER-1 — Proof Types Model

## Purpose
Define the allowed proof types and their required fields so proof checking is deterministic.

## What a proof is
A proof is a structured record that a verification action occurred and produced evidence.

## Proof type examples
- command_run (a command executed, with logs/ref)
- test_suite (unit/integration/e2e results)
- lint_check (lint/format verification)
- build_artifact (successful build output)
- security_scan (SAST/DAST/dependency scan)
- manual_attestation (human approval for something)

## Required proof fields (minimum)
- proof_id
- proof_type
- status (pass/fail/warn)
- evidence_refs (paths to logs/artifacts)
- recorded_at
```

Piece B — Proof types registry schema (JSON Schema)

File: `Axion/libraries/verification/schemas/proof_types.v1.schema.json`

```
{
```

```
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "proof_types.v1",
  "type": "object",
  "required": ["registry_id", "schema_version", "types"],
  "properties": {
    "registry_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "types": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/proof_type" }
    }
  },

  "$defs": {
    "proof_type": {
      "type": "object",
      "required": ["proof_type", "required_fields", "description"],
      "properties": {
        "proof_type": {
          "type": "string",
          "enum": ["command_run", "test_suite", "lint_check", "build_artifact", "security_scan",
"manual_attestation"]
        },
        "description": { "type": "string", "minLength": 5 },
        "required_fields": { "type": "array", "items": { "type": "string" }, "minItems": 1 }
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Starter proof types registry

File: `Axion/libraries/verification/registries/proof_types.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "proof_types.v1",
```

```
  "types": [
    {
      "proof_type": "command_run",
      "description": "Evidence that a command was executed with recorded outputs/log refs.",
      "required_fields": ["command_run_id", "command", "exit_code", "logs_ref"]
    },
    {
      "proof_type": "test_suite",
      "description": "Evidence that a test suite ran and produced results.",
      "required_fields": ["suite", "passed", "failed", "results_ref"]
    },
    {
      "proof_type": "lint_check",
      "description": "Evidence that lint/format checks ran.",
      "required_fields": ["tool", "status", "logs_ref"]
    },
    {
      "proof_type": "build_artifact",
      "description": "Evidence that a build produced an artifact.",
      "required_fields": ["artifact_kind", "artifact_ref"]
    },
    {
      "proof_type": "security_scan",
      "description": "Evidence that a security scan ran (SAST/DAST/deps).",
      "required_fields": ["scan_type", "status", "report_ref"]
    },
    {
      "proof_type": "manual_attestation",
      "description": "Human attestation/approval for a requirement.",
      "required_fields": ["actor_id", "role", "statement", "expires_at"]
    }
  ]
}
```

Piece D — Determinism rules

File: `Axion/libraries/verification/VER-1_determinism_rules.md`

---
library: verification
id: VER-1a
schema_version: 1.0.0

status: draft

VER-1a — Determinism Rules (Proof Types)

- proof_type is a closed enum and must exist in the registry.
- required_fields are enforced by proof ledger validation.
- Registry is pinned per run (for reproducibility).

Piece E — Validation checklist

File: `Axion/libraries/verification/VER-1_validation_checklist.md`

---
library: verification
id: VER-1b
schema_version: 1.0.0
status: draft

VER-1b — Validation Checklist

- [ ] proof_types registry validates against schema
- [ ] proof_type values are unique and from the allowed enum set
- [ ] each proof_type has required_fields and description
- [ ] registry is pinned in the run manifest (or via pins)

Command run tracking schema

VER-3 — Command run tracking schema (what ran, where, result)

Piece A — Command run tracking model (concept)

File: `Axion/libraries/verification/VER-3_command_run_model.md`

```
---
library: verification
id: VER-3
schema_version: 1.0.0
status: draft
---

# VER-3 — Command Run Tracking Model

## Purpose
Track execution of verification commands in a structured way so proofs can reference:
- what command ran
- in what environment (adapter profile)
- what the result was (exit code/status)
- where logs/output artifacts live

This enables reproducibility, auditing, and evidence collection without dumping logs into gate reports.

## Key properties
- command runs are run-scoped
- logs are referenced, not embedded
- results are minimal but sufficient for proof generation
```

Piece B — Command run schema (JSON Schema)

File: `Axion/libraries/verification/schemas/command_run.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "command_run.v1",
  "type": "object",
```

```
"required": [
  "command_run_id",
  "run_id",
  "command",
  "cwd",
  "adapter_profile_id",
  "status",
  "exit_code",
  "started_at",
  "ended_at",
  "logs_ref"
],
"properties": {
  "command_run_id": { "type": "string", "pattern": "^CMD-[A-Z0-9]{6,}$" },
  "run_id": { "type": "string" },

  "command": { "type": "string", "minLength": 1 },
  "cwd": { "type": "string", "minLength": 1 },

  "adapter_profile_id": { "type": "string" },

  "status": { "type": "string", "enum": ["succeeded", "failed", "skipped"] },
  "exit_code": { "type": "integer" },

  "started_at": { "type": "string", "format": "date-time" },
  "ended_at": { "type": "string", "format": "date-time" },

  "stdout_ref": { "$ref": "#/$defs/ref" },
  "stderr_ref": { "$ref": "#/$defs/ref" },
  "logs_ref": { "$ref": "#/$defs/ref" },

  "artifacts_produced": {
    "type": "array",
    "items": { "$ref": "#/$defs/ref" }
  }
},

"$defs": {
  "ref": {
    "type": "object",
    "required": ["path"],
    "properties": {
      "path": { "type": "string" },
      "hash": { "type": "string" }
```

```
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Command runs registry schema (all runs for a pipeline run)

File: `Axion/libraries/verification/schemas/command_run_log.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "command_run_log.v1",
  "type": "object",
  "required": ["log_id", "run_id", "runs"],
  "properties": {
    "log_id": { "type": "string", "pattern": "^CMDLOG-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },
    "runs": {
      "type": "array",
      "items": { "$ref": "command_run.v1.schema.json" }
    }
  },
  "additionalProperties": false
}
```

Piece D — Determinism rules (command tracking)

File: `Axion/libraries/verification/VER-3_determinism_rules.md`

```
---
library: verification
id: VER-3a
schema_version: 1.0.0
status: draft
---
```

VER-3a — Determinism Rules (Command Runs)

- command_run_id is generated deterministically per (run_id + command + cwd + started_at bucket) OR generated sequentially but recorded.
- command strings must be exact (no hidden expansions) and stored verbatim.
- logs refs are stable paths; content is not embedded.
- command runs are ordered deterministically in command_run_log for serialization:
  - started_at
  - command_run_id

Piece E — Validation checklist

File: `Axion/libraries/verification/VER-3_validation_checklist.md`

---
library: verification
id: VER-3b
schema_version: 1.0.0
status: draft

VER-3b — Validation Checklist

- [ ] command_run validates against schema
- [ ] command_run_log validates against schema
- [ ] logs_ref present for each non-skipped run
- [ ] adapter_profile_id recorded
- [ ] deterministic ordering applied in command_run_log

Completion criteria

VER-4 — Completion criteria (unit done + run done)

Piece A — Completion model (concept)

File: `Axion/libraries/verification/VER-4_completion_model.md`

---
library: verification
id: VER-4
schema_version: 1.0.0
status: draft

VER-4 — Completion Criteria Model

Purpose
Define what “done” means at two levels:
1) **Unit done**: a work item / acceptance criterion is complete
2) **Run done**: the overall run is complete and releasable

Inputs
- Acceptance map (required acceptance criteria + evidence requirements)
- Proof ledger (what proofs exist)
- Gate reports (which gates passed/failed)
- Standards snapshot (may introduce required proofs)

Key concept
Completion is enforceable only if it is:
- explicit (rules not vague)
- measurable (proof requirements and artifact contract IDs)
- deterministic (no subjective evaluation unless manual_attestation is used)

Piece B — Completion criteria schema (JSON Schema)

File: `Axion/libraries/verification/schemas/completion_criteria.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
```

```
"schema_version": "1.0.0",
"id": "completion_criteria.v1",
"type": "object",
"required": ["criteria_id", "schema_version", "unit_done", "run_done"],
"properties": {
  "criteria_id": { "type": "string", "minLength": 3 },
  "schema_version": { "type": "string", "minLength": 1 },

  "unit_done": {
    "type": "object",
    "required": ["requires"],
    "properties": {
      "requires": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/requirement" }
      }
    },
    "additionalProperties": false
  },

  "run_done": {
    "type": "object",
    "required": ["requires"],
    "properties": {
      "requires": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/requirement" }
      }
    },
    "additionalProperties": false
  }
},

"$defs": {
  "requirement": {
    "type": "object",
    "required": ["kind", "ref"],
    "properties": {
      "kind": { "type": "string", "enum": ["proof", "artifact", "gate_pass", "manual_attestation"] },
      "ref": { "type": "string", "minLength": 2 },
      "notes": { "type": "string" }
    },
```

```
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Starter completion criteria (baseline)

File: `Axion/libraries/verification/registries/completion_criteria.v1.json`

```
{
  "schema_version": "1.0.0",
  "criteria_id": "completion_criteria.v1",
  "unit_done": {
    "requires": [
      { "kind": "proof", "ref": "proof:required_for_work_item" }
    ]
  },
  "run_done": {
    "requires": [
      { "kind": "gate_pass", "ref": "G1_INTAKE_VALIDITY" },
      { "kind": "gate_pass", "ref": "G2_CANONICAL_INTEGRITY" },
      { "kind": "gate_pass", "ref": "G3_STANDARDS_RESOLVED" },
      { "kind": "gate_pass", "ref": "G4_TEMPLATE_SELECTION" },
      { "kind": "gate_pass", "ref": "G5_TEMPLATE_COMPLETENESS" },
      { "kind": "gate_pass", "ref": "G6_PLAN_COVERAGE" },
      { "kind": "gate_pass", "ref": "G7_VERIFICATION" },

      { "kind": "artifact", "ref": "KIT_MANIFEST" },
      { "kind": "artifact", "ref": "PROOF_LEDGER" }
    ]
  }
}
```

Piece D — Determinism rules (completion evaluation)

File: `Axion/libraries/verification/VER-4_determinism_rules.md`

---

library: verification
id: VER-4a
schema_version: 1.0.0
status: draft

VER-4a — Determinism Rules (Completion)

- Completion evaluation reads only pinned artifacts:
  - acceptance_map, proof_ledger, gate_report(s), run_manifest
- Manual decisions must be represented as manual_attestation proofs (no implicit judgment).
- The criteria registry is pinned per run.
- Missing requirements lists are deterministically ordered.

Piece E — Validation checklist

File: `Axion/libraries/verification/VER-4_validation_checklist.md`

---
library: verification
id: VER-4b
schema_version: 1.0.0
status: draft

VER-4b — Validation Checklist

- [ ] completion_criteria validates against schema
- [ ] unit_done and run_done contain measurable requirements
- [ ] manual decisions require manual_attestation proof type
- [ ] evaluation depends only on pinned artifacts and registries

Gates

VER-5 — Verification command policy (allowed commands + environments)

Piece A — Command policy model (concept)

File: `Axion/libraries/verification/VER-5_command_policy_model.md`

```
---
library: verification
id: VER-5
schema_version: 1.0.0
status: draft
---

# VER-5 — Verification Command Policy Model

## Purpose
Define what verification commands are allowed to run in which environments.
This prevents unsafe or non-reproducible verification.

## Inputs
- adapter_profile (capabilities, command policy id) from system/
- verification policy registry (this library)
- run profile + risk class

## Outputs
- allow/deny decision for a command run request
- recorded decision (policy decision id + reason) for audit
```

Piece B — Verification command policy schema

File: `Axion/libraries/verification/schemas/verification_command_policy.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "verification_command_policy.v1",
  "type": "object",
```

```
  "required": ["policy_id", "schema_version", "rules"],
  "properties": {
    "policy_id": { "type": "string", "pattern": "^VERPOL-[A-Z0-9]{6,}$" },
    "schema_version": { "type": "string", "minLength": 1 },

    "rules": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["rule_id", "match", "decision"],
        "properties": {
          "rule_id": { "type": "string", "pattern": "^VRULE-[A-Z0-9_\\-]+$" },

          "match": {
            "type": "object",
            "required": ["pattern"],
            "properties": {
              "pattern": { "type": "string", "minLength": 1, "description": "Glob/regex pattern for
command string" },
              "profiles": { "type": "array", "items": { "type": "string" } },
              "risk_classes": { "type": "array", "items": { "type": "string" } },
              "adapter_types": { "type": "array", "items": { "type": "string" } }
            },
            "additionalProperties": false
          },

          "decision": {
            "type": "object",
            "required": ["outcome"],
            "properties": {
              "outcome": { "type": "string", "enum": ["allow", "deny", "require_approval"] },
              "reason": { "type": "string" }
            },
            "additionalProperties": false
          }
        },
        "additionalProperties": false
      }
    }
  },
  "additionalProperties": false
}
```

Piece C — Starter verification command policy (baseline safe set)

File:
`Axion/libraries/verification/registries/verification_command_policy.v1.json`


```
{
  "schema_version": "1.0.0",
  "policy_id": "VERPOL-BASE01",
  "rules": [
    {
      "rule_id": "VRULE-ALLOW-NPM-TEST",
      "match": {
        "pattern": "^npm\\s+test(\\s+.*)?$",
        "profiles": ["PROFILE-API", "PROFILE-WEB", "PROFILE-MOBILE"],
        "risk_classes": ["PROTOTYPE", "PROD", "COMPLIANCE"],
        "adapter_types": ["local", "replit", "ci"]
      },
      "decision": { "outcome": "allow", "reason": "Standard test execution." }
    },
    {
      "rule_id": "VRULE-ALLOW-NPM-LINT",
      "match": {
        "pattern": "^npm\\s+run\\s+(lint|format)(\\s+.*)?$",
        "profiles": ["PROFILE-API", "PROFILE-WEB", "PROFILE-MOBILE"],
        "risk_classes": ["PROTOTYPE", "PROD", "COMPLIANCE"],
        "adapter_types": ["local", "replit", "ci"]
      },
      "decision": { "outcome": "allow", "reason": "Lint/format verification." }
    },
    {
      "rule_id": "VRULE-DENY-DESTRUCTIVE",
      "match": {
        "pattern": "^(rm\\s+-rf|del\\s+/s|shutdown|reboot|mkfs).*",
        "profiles": ["PROFILE-API", "PROFILE-WEB", "PROFILE-MOBILE", "PROFILE-INFRA",
"PROFILE-DATA", "PROFILE-MIXED"],
        "risk_classes": ["PROTOTYPE", "PROD", "COMPLIANCE"],
        "adapter_types": ["local", "replit", "ci", "container"]
      },
      "decision": { "outcome": "deny", "reason": "Destructive commands are forbidden." }
    }
  ]
```

}

Piece D — Determinism rules (policy evaluation)

File: `Axion/libraries/verification/VER-5_determinism_rules.md`

```
---
library: verification
id: VER-5a
schema_version: 1.0.0
status: draft
---

# VER-5a — Determinism Rules (Command Policy)

- Rules are evaluated in file order; first match wins.
- Command string is matched verbatim (no shell expansion assumed).
- Policy is pinned per run.
- require_approval outcome triggers policy hook and audit record.
```

Piece E — Validation checklist

File: `Axion/libraries/verification/VER-5_validation_checklist.md`

```
---
library: verification
id: VER-5b
schema_version: 1.0.0
status: draft
---

# VER-5b — Validation Checklist

- [ ] verification_command_policy validates against schema
- [ ] rule evaluation order is deterministic (first match wins)
- [ ] deny patterns are present for destructive commands
- [ ] policy is pinned per run
- [ ] require_approval integrates with policy hooks + audit
```

Minimum viable verification

VER-7 — Minimum viable verification set (runtime-required files + “done”)

Piece A — MV verification inventory

File: `Axion/libraries/verification/VER-7_minimum_viable_set.md`

---
library: verification
id: VER-7
schema_version: 1.0.0
status: draft

VER-7 — Minimum Viable verification/ Set

Goal
Define the smallest verification library set required to record proofs and enforce run completion deterministically.

Required runtime files (minimum)
1) Core docs
- VER-0_purpose.md
- VER-0_boundary_checklist.md
- VER-1_proof_types_model.md
- VER-1_determinism_rules.md
- VER-1_validation_checklist.md
- VER-2_proof_ledger_model.md
- VER-2_determinism_rules.md
- VER-2_validation_checklist.md
- VER-3_command_run_model.md
- VER-3_determinism_rules.md
- VER-3_validation_checklist.md
- VER-4_completion_model.md
- VER-4_determinism_rules.md
- VER-4_validation_checklist.md
- VER-5_command_policy_model.md
- VER-5_determinism_rules.md
- VER-5_validation_checklist.md
- VER-6_verification_gates.md
- VER-6_gate_mapping.md
- VER-6_evidence_requirements.md

- VER-6_determinism_rules.md
- VER-6_validation_checklist.md
- VER-7_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/proof_types.v1.schema.json
- schemas/proof_ledger.v1.schema.json
- schemas/command_run.v1.schema.json
- schemas/command_run_log.v1.schema.json
- schemas/completion_criteria.v1.schema.json
- schemas/verification_command_policy.v1.schema.json

3) Registries (runtime defaults)
- registries/proof_types.v1.json
- registries/completion_criteria.v1.json
- registries/verification_command_policy.v1.json

4) Gate specs (optional)
- VER-6_verification_gates.spec.json

Piece B — Definition of done (verification/)

File: `Axion/libraries/verification/VER-7_definition_of_done.md`

---
library: verification
id: VER-7a
schema_version: 1.0.0
status: draft

VER-7a — verification/ Definition of Done

verification/ is “done” when:

Schemas + registries
- [ ] All verification schemas validate (JSON Schema check)
- [ ] proof_types registry validates and is pinned
- [ ] completion_criteria registry validates and is pinned
- [ ] verification_command_policy registry validates and is pinned

Runtime behavior (contract-level)

- [ ] Command runs can be recorded with logs refs (no embedded logs)
- [ ] Proofs can be recorded and validated by proof_type required fields
- [ ] Proof ledger is append-only and references evidence by stable paths
- [ ] Completion criteria can be evaluated deterministically from pinned artifacts

Gates
- [ ] VER-GATE-01..07 implemented and mapped to G7_VERIFICATION
- [ ] Missing proofs/artifacts produce actionable evidence + remediation
- [ ] Command policy violations are detected and blocked or require approval

---

Piece C — Minimal folder tree

File: `Axion/libraries/verification/VER-7_minimal_tree.txt`

```
Axion/libraries/verification/
  VER-0_purpose.md
  VER-0_boundary_checklist.md
  VER-1_proof_types_model.md
  VER-1_determinism_rules.md
  VER-1_validation_checklist.md
  VER-2_proof_ledger_model.md
  VER-2_determinism_rules.md
  VER-2_validation_checklist.md
  VER-3_command_run_model.md
  VER-3_determinism_rules.md
  VER-3_validation_checklist.md
  VER-4_completion_model.md
  VER-4_determinism_rules.md
  VER-4_validation_checklist.md
  VER-5_command_policy_model.md
  VER-5_determinism_rules.md
  VER-5_validation_checklist.md
  VER-6_verification_gates.md
  VER-6_verification_gates.spec.json
  VER-6_gate_mapping.md
  VER-6_evidence_requirements.md
  VER-6_determinism_rules.md
  VER-6_validation_checklist.md
  VER-7_minimum_viable_set.md
  VER-7_definition_of_done.md
  VER-7_minimal_tree.txt
```

```
schemas/
  proof_types.v1.schema.json
  proof_ledger.v1.schema.json
  command_run.v1.schema.json
  command_run_log.v1.schema.json
  completion_criteria.v1.schema.json
  verification_command_policy.v1.schema.json

registries/
  proof_types.v1.json
  completion_criteria.v1.json
  verification_command_policy.v1.json
```

Say **“next”** to start the next library folder. Recommended next is **kit/** (KIT-0).